스마트팜 시스템 소프트웨어 공학 분석 보고서

1. 개요

본 보고서는 스마트팜 모니터링 시스템의 소프트웨어 공학적 설계 및 구현을 분석이다. 이 시스템은 센서 데이터 수집, 처리, 웹 기반 모니터링을 제공하는 IoT 시스템이다다

2. 요구사항 분석 (Requirements Analysis)

2.1 기능적 요구사항 (Functional Requirements)

주요 기능

- 센서 데이터 실시간 수집 (온도, 습도, 토양수분, 광도)
- 자동 제어 기능 (워터펌프, LED 조명)
- 웹 기반 모니터링 대시보드
- 외부 디바이스(라즈베리파이)와의 통신
- 알림 시스템

사용자 스토리

- 사용자는 웹 브라우저를 통해 실시간 센서 데이터를 확인할 수 있다
- 시스템은 토양이 건조하면 자동으로 워터펌프를 작동시킨다
- 라즈베리파이는 센서 데이터를 서버로 전송할 수 있다

2.2 비기능적 요구사항 (Non-functional Requirements)

성능 요구사항

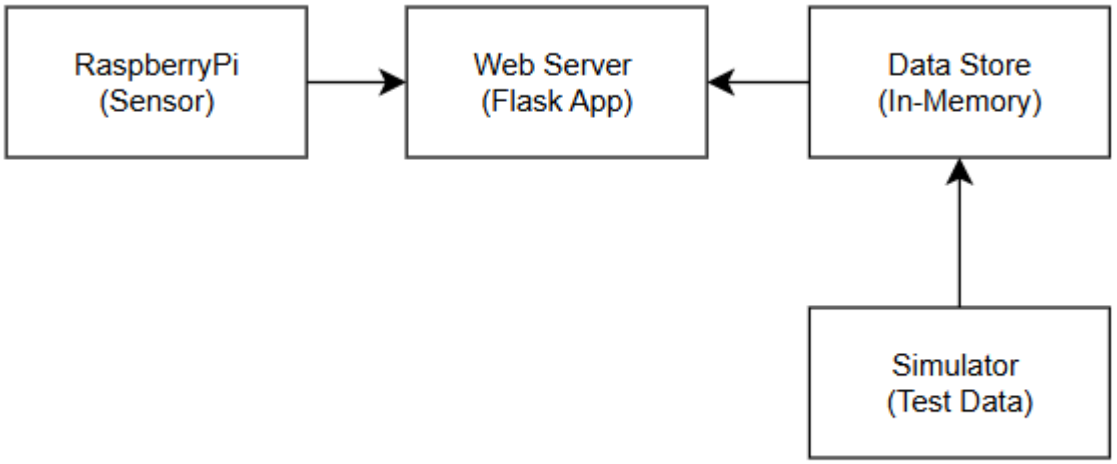
- 센서 데이터 업데이트 주기: 5초
- 외부 디바이스 통신 주기: 10초
- 웹 응답 시간: 1초 이내

확장성 요구사항

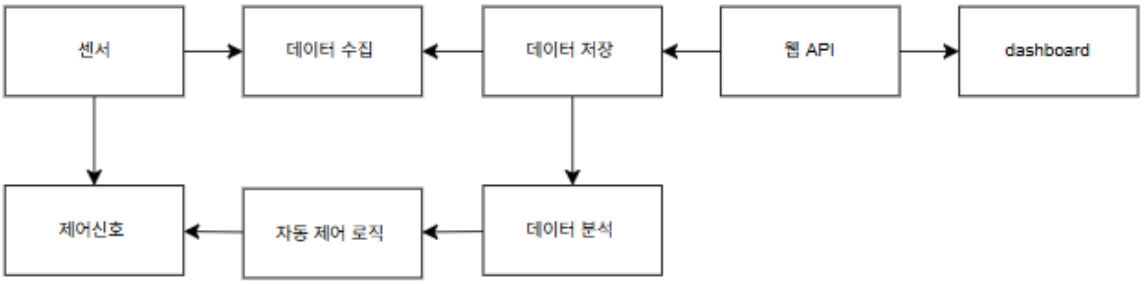
- 다중 센서 노드 지원 가능한 구조
- RESTful API를 통한 확장성

3. 시스템 추상화 (System Abstraction)

3.1 시스템 아키텍처



3.2 데이터 흐름 다이어그램



3.3 시스템 구조 흐름도

1. 시스템 시작
   DataStore 초기화 (기본값 설정)
   Simulator 시작 (백그라운드 스레드)
   WebApp 시작 (Flask 서버)

2. 런타임 동작
   Simulator: 5초마다 가상 데이터 생성 → DataStore 업데이트
   WebApp: 브라우저 요청 처리 → DataStore에서 데이터 조회
   RaspberryPi: 10초마다 실제 센서 데이터 → WebApp API로 전송

3. 사용자 경험
   브라우저에서 대시보드 접속
   JavaScript가 주기적으로 /api/data 호출
   실시간으로 업데이트되는 센서 값 확인

4. 모듈화 및 데이터 통신 방식

4.1 모듈 구조

Layer 1: 데이터 계층

- `SimpleDataStore`: 중앙집중식 데이터 관리

Layer 2: 로직 계층

- `SimpleSimulator`: 테스트 데이터 생성
- `RaspberryPiSensor`: 센서 데이터 수집

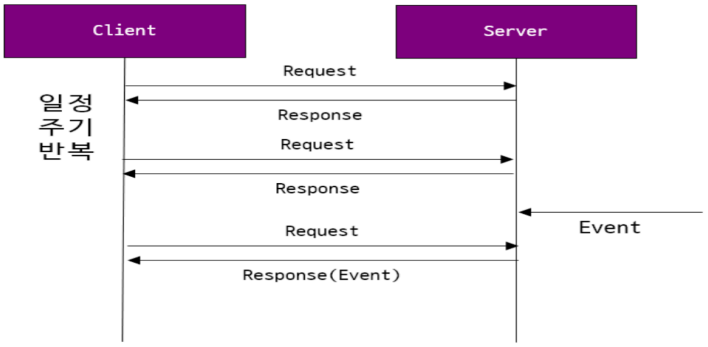
Layer 3: 웹 계층

- `SimpleWebApp`: 웹 인터페이스 제공

4.2 모듈 간 의존성

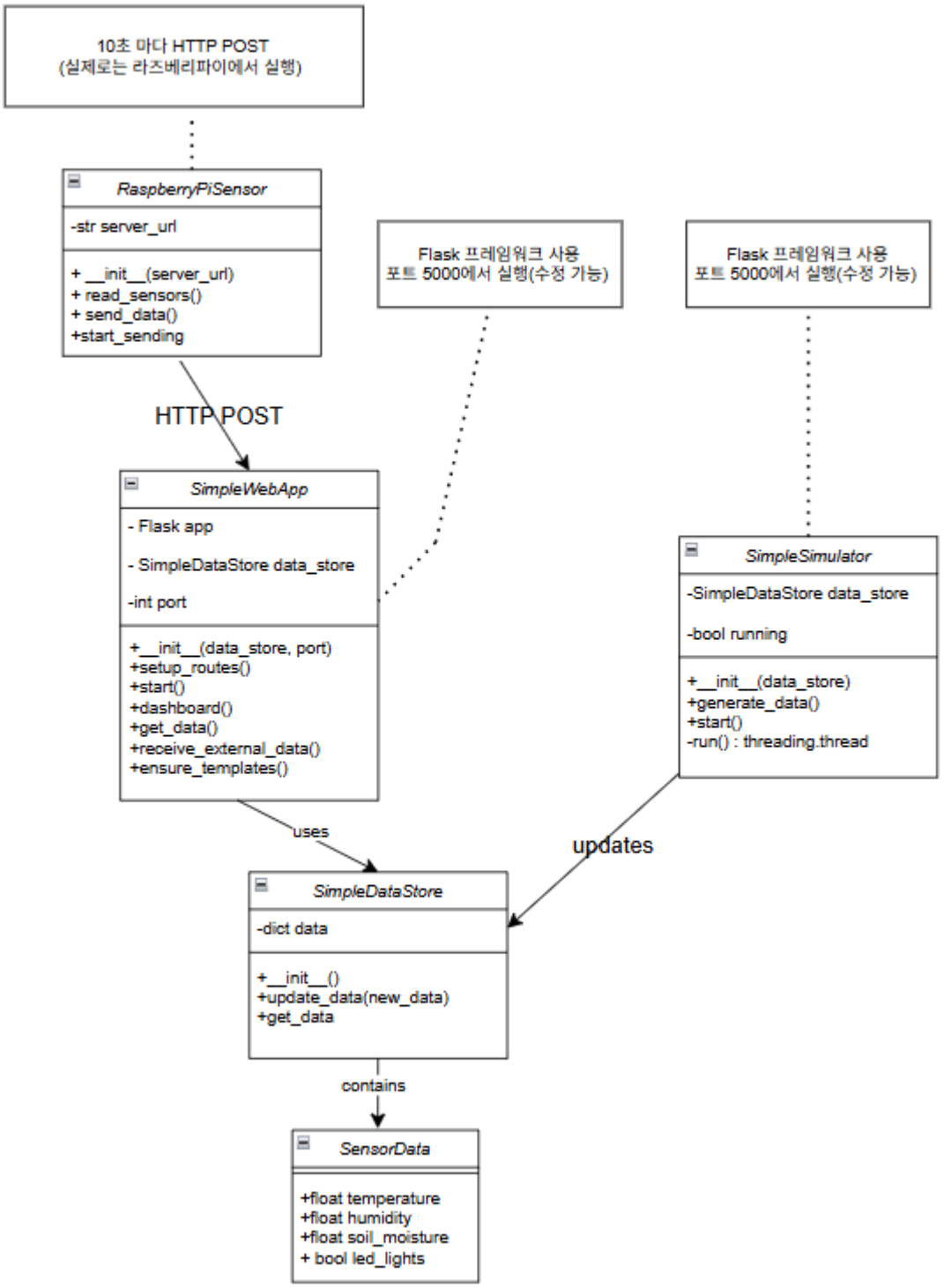
SimpleWebApp -> SimpleDataStore <- SimpleSimulator
↑
RaspberryPiSensor (HTTP 통신)

4.3 데이터 통신 방식 : Polling

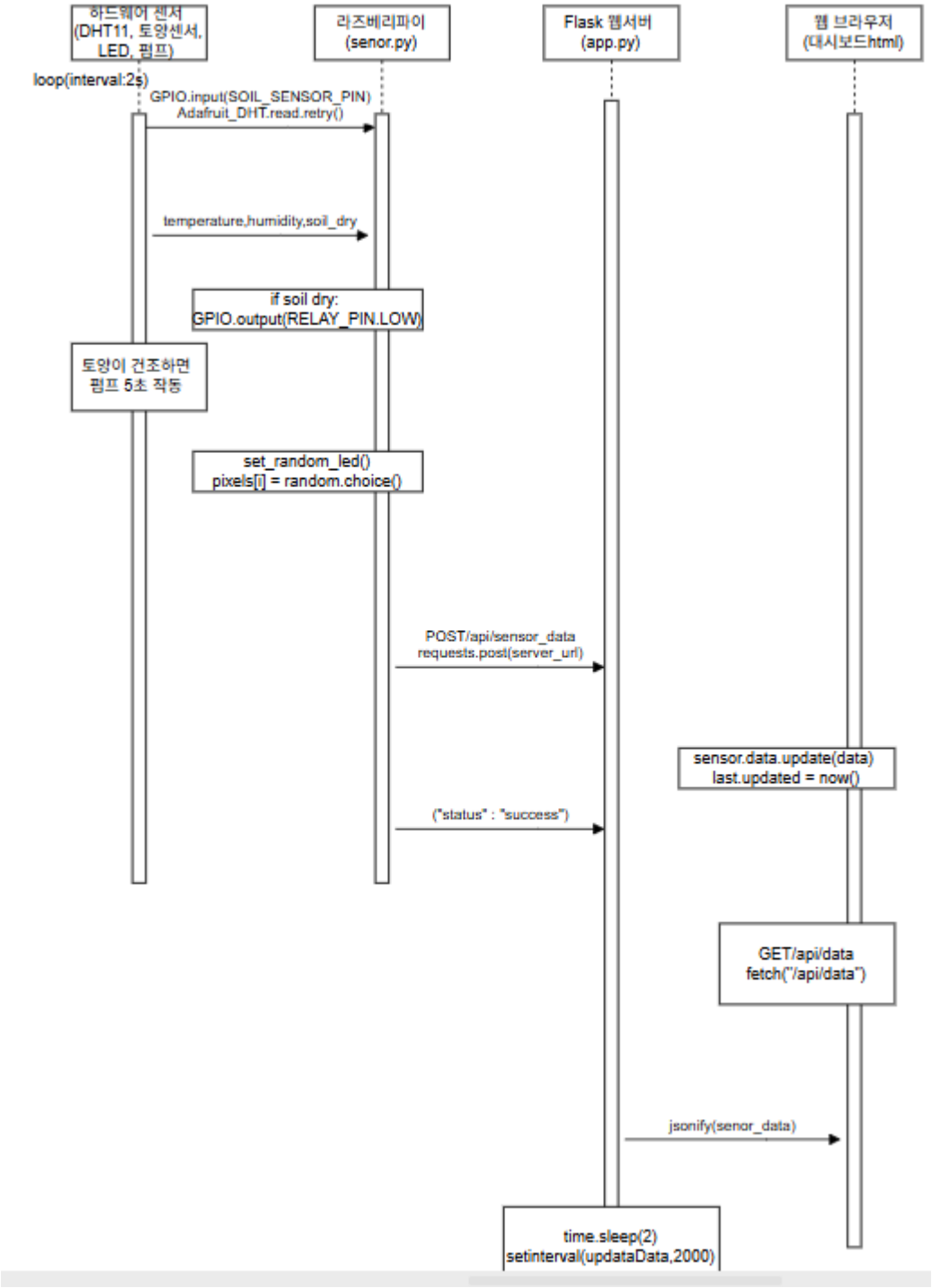


HTTP 프로토콜을 통해 실시간 통신처럼 보이게끔 하기 위해 이 방식을 사용했다.

4.3 모듈 동작 과정**(**시뮬레이션으로 구상**)**


10초 마다 HTTP POST
(실제로는 라즈베리파이에서 실행)
RaspberryPiSensor
-str server_url
+ __init__(server_url)
+ read_sensors()
+ send_data()
+start_sending
Flask 프레임워크 사용
포트 5000에서 실행(수정 가능)
Flask 프레임워크 사용
포트 5000에서 실행(수정 가능)
HTTP POST
SimpleWebApp
- Flask app
- SimpleDataStore data_store
-int port
+__init__(data_store, port)
+setup_routes()
+start()
+dashboard()
+get_data()
+receive_external_data()
+ensure_templates()
SimpleSimulator
-SimpleDataStore data_store
-bool running
+__init__(data_store)
+generate_data()
+start()
-run() : threading.thread
uses
updates
SimpleDataStore
-dict data
+__init__()
+update_data(new_data)
+get_data
contains
SensorData
+float temperature
+float humidity
+float soil_moisture
+ bool led_lights

4.4 모듈 동작 과정 (실제)



설명:

1. 센서 데이터 수집: **GPIO**를 통해 토양 수분, 온습도 측정
2. 자동 제어: 토양이 건조하면 펌프 자동 작동 **(5**초**)**
3. **LED** 표시: 랜덤 색상으로 상태 표시

4. 서버 전송**: HTTP POST**로 **Flask** 서버에 데이터 전송

5. 웹 조회**:** 브라우저가 **2**초마다 최신 데이터 요청

6. 실시간 표시**:** 대시보드에 센서 상태 실시간 업데이트

5. 구체화 및 구현 분석 (Implementation Analysis)

5.1 클래스별 상세 분석

SimpleDataStore 클래스

역할: 중앙집중식 데이터 저장소 핵심 기능:

- `update_data()`: 센서 데이터 업데이트 및 타임스탬프 관리
- `get_data()`: 현재 데이터 상태 반환

설계 패턴:: Thread-safe하지

```
# 데이터 구조 예시
{
   "temperature": 25.0,
   "humidity": 60.0,
   "soil_moisture": 45.0,
   "light_intensity": 500,
   "water_pump": False,
   "led_lights": False,
   "alerts": [],
   "last_updated": "2025-05-31T...",
   "source": "simulation"
}
```

SimpleSimulator 클래스

역할: 개발/테스트용 모의 센서 데이터 생성 핵심 기능:

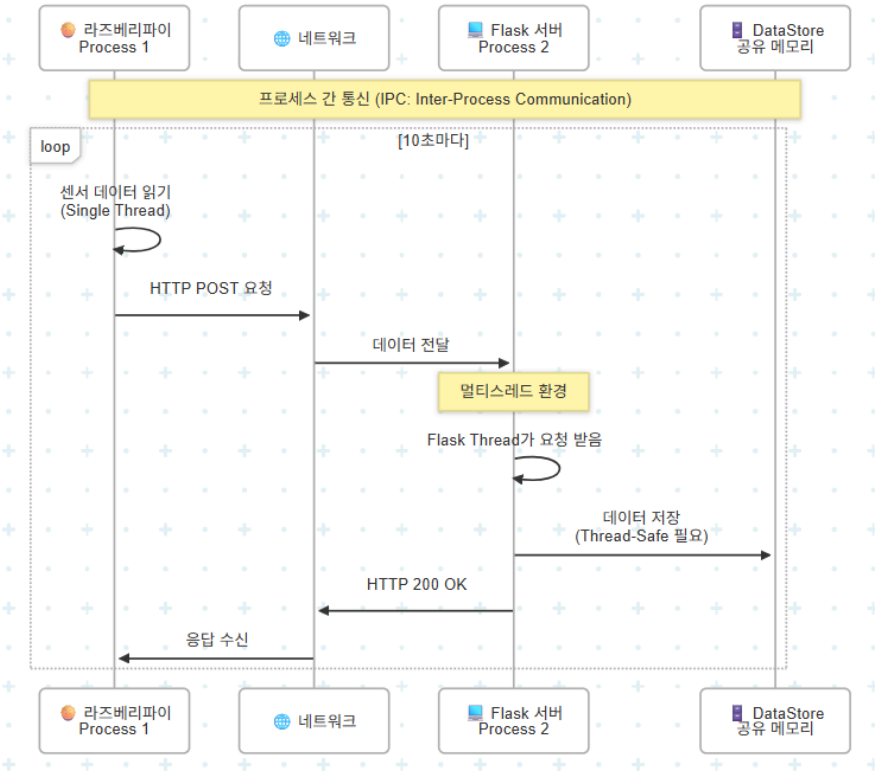
- `generate_data()`: 실제적인 센서 값 시뮬레이션
- `start()`: 백그라운드 스레드로 지속적 데이터 생성

스레딩: `threading.Thread`를 사용한 비동기 처리로 5초마다 센서 데이터를 생성하게 하므로서 test code가 잘 동작하는지 확인 할 수 있었다.

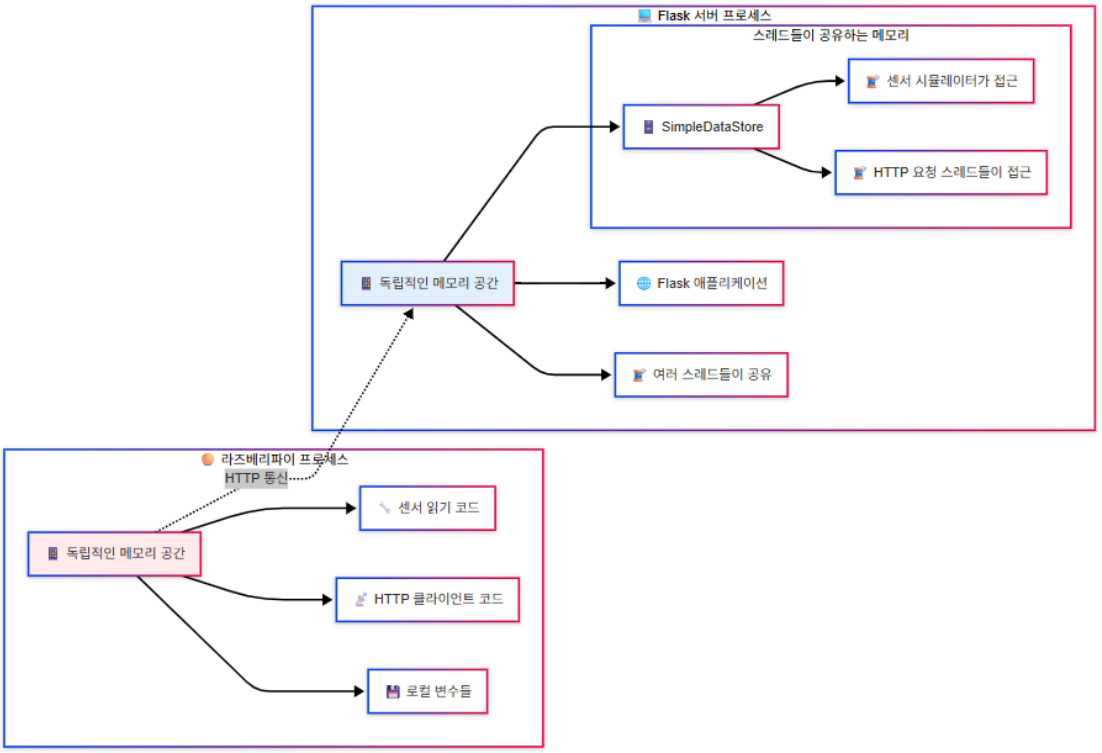
실시간 통신 처럼 보이기 위해 CPU의 다중 스레드 이용을 사용했다.

1. 라즈베리파이: 단일 스레드 -> 보통 센서를 읽고 대부분 대기
2. 서버: 멀티 스레드 -> 여러 작업 동시 처리
3. 메모리: 프로세스 간 독립, 스레드 간 데이터는 공유

프로세스 간 통신 구조:



메모리 간 구조:



SimpleWebApp 클래스

역할: 웹 기반 사용자 인터페이스 제공 핵심 기능:

- `setup_routes()`: REST API 엔드포인트 정의
- `start()`: Flask 웹 서버 시작

API 엔드포인트:

- `GET /`: 메인 대시보드 (HTML)
- `GET /api/data`: 센서 데이터 조회 (JSON)
- `POST /api/external_data`: 외부 디바이스 데이터 수신

설계 패턴: MVC 패턴의 Controller 역할

RaspberryPiSensor 클래스

역할: 외부 센서 디바이스 시뮬레이션 핵심 기능:

- `read_sensors()`: 센서 데이터 읽기 (시뮬레이션)
- `send_data()`: HTTP POST를 통한 데이터 전송

- `start_sending()`: 주기적 데이터 전송

통신 프로토콜: HTTP/JSON 에러 처리: Try-catch를 통한 네트워크 오류 처리

6. 장점과 개선점

.1 현재 구현의 장점

단순성: 학습과 이해가 쉬운 구조 모듈화: 각 기능이 독립적인 클래스로 분리 확장성: RESTful API를 통한 확장 가능 테스트 용이성: 시뮬레이터를 통한 개발 환경 구축

6.2 개선 필요사항

Thread Safety: 동시성 문제 해결 필요

Race condition: 여러 process/Thread가 동시에 같은 데이텅를 조작할 때 타이밍,접근 순서에 따라 결과가 달라 질 수도 있다.

데이터 영속성: 메모리 기반에서 데이터베이스로 확장

SQLite 또는 다른 데이터베이스 사용 고려

에러 처리: 더 체계적인 예외 처리 보안: 인증 및 권한 관리 설정 관리: 하드코딩된 값들을 설정 파일로 분리

작성자 **: 20223866** 정재민
작성일: 2025-05-31

참고 자료 :

1. 멀티 스레딩 vs 스레딩 https://inpa.tistory.com/entry/%F0%9F%91%A9%E2%80%8D%F0%9F%92%BB-multi-process-multi-thread
2. 멀티 스레딩 설명 https://velog.io/@turningtwenty/%EC%9A%B4%EC%98%81%EC%B2%B4%EC%A0%9COperating-System-%EC%8A%A4%EB%A0%88%EB%93%9C%EC%99%80-%EB%A9%80%ED%8B%B0%EC%8A%A4%EB%A0%88%EB%94%A9
3. https://engineerinsight.tistory.com/281#google_vignette
4. 폴링 방식 https://usingsystem.tistory.com/45
5. 웹 훅 방식 https://docs.tosspayments.com/resources/glossary/webhook
6. 소켓 방식

https://helloworld-88.tistory.com/215

7. 소켓,포트,TCP 통신 방식
   https://www.youtube.com/watch?v=X73Jl2nsqiE&list=WL&index=8&t=1026s
8. HTTP 통신 방법
   https://www.youtube.com/watch?v=GxCf1SsLVB4
9. REST API
   https://www.youtube.com/watch?v=fB3MB8TXNXM
10. Flask -route
    https://www.youtube.com/watch?v=iLBwWFrxpV0
11. 스레딩 사용 예제
    https://miki3079.tistory.com/50
12. python design pattern
    https://python-patterns.guide/
13. 스레딩 기반 병렬 처리
    https://docs.python.org/3/library/threading.html
14. 병렬 작업
    https://docs.python.org/3/library/concurrent.futures.html
15. flask
    https://flask.palletsprojects.com/en/stable/
16. Restful API
    https://restfulapi.net/
17. Python connect api
    https://realpython.com/python-api/